

1. Why the approximate KL is valid?

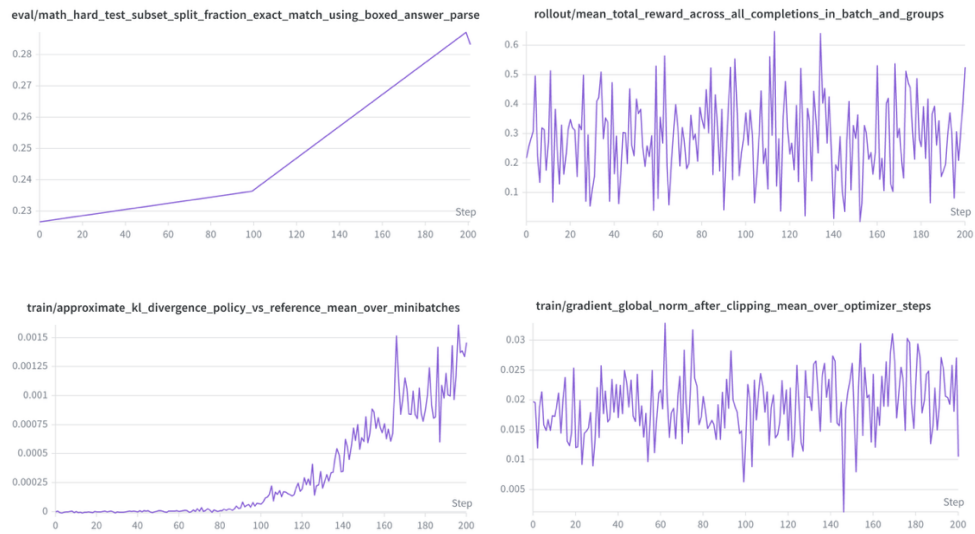
As proved in the assignment instruction, the expectation of the estimator over actions strictly equals to the expectation of original KL divergence at each token position. Therefore, the approximate KL is valid as it's an unbiased estimator. What's more, the estimator is always non-negative and equals zero iff the reference probability equals to the probability of current policy. Intuitively this is exactly punishing the policy when its output deviates the reference policy. Computing the exact KL would be much more expensive because at every token position it would require summing over the entire vocabulary distribution for both policies, rather than using only the sampled token's log-probabilities, which increases both compute and memory cost.

2. Implementation Process

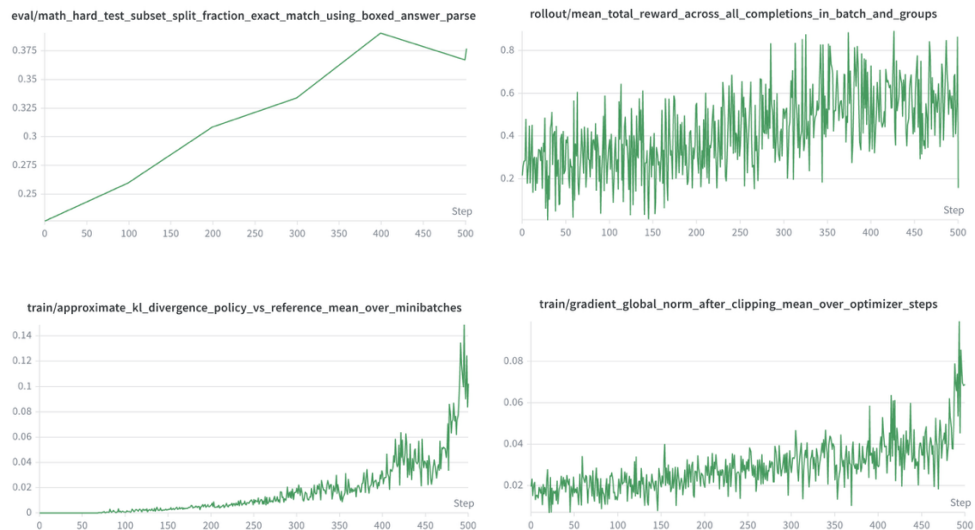
- a) Firstly, I implemented the function that computes log probability w.r.t every token in the batch. The function takes in batches of tokenized sequences, computes the corresponding logits, and finally gets the log probabilities of each token using cross-entropy function.
- b) Then I built the completion mask by setting the bits at positions smaller than prompt input length in attentions mask to be 0, and shift the mask left by 1 to match the shape of per token log probability tensor.
- c) After that I implemented the function approximating KL divergence by averaging the estimator result w.r.t every unmasked token in the batch.
- d) The next thing I did is to slice the rollout batch into several minibatches for training.
- e) Then I calculated the advantage of each sampled completion in the batch by comparing its reward with others in the group and normalized the advantages within the group and within the whole batch to stabilize training.
- f) Next, I implemented the GR-REINFORCE algorithm by calculating per token log probability, per completion averaged log probability, averaged advantage times per completion averaged log probability, estimated KL divergence and the entropy. The final loss function consists of the third and the fourth term.
- g) Finally, I completed the full GRPO pipeline. It first calculates the probability ratio per token. Then it multiplies the advantage per completion to both original probability ratio per token tensor and the clipped version. The final advantage per token is the minimum of both two tensors. Finally, the pipeline averages the values of the whole tensor and added it to the estimated KL divergence to get the final loss function. In addition, the clipped ratio is recorded as well.
- h) **One confusion point:** when implementing GRPO, the comment suggests $\{\text{per_token_obj_t} = \min(\text{unclipped_t}, \text{clipped_t}) * \text{mask_t}\}$, which sets the value in the tensor to be zero where the mask is 0. But later when summing the values across the completions, we use $\{\text{masked_mean_per_row}(\dots)\}$, which itself already ignores the masked position. So, the previous “* mask_t” seems duplicated and there's no need to multiply it.

3. GR-REINFORCE v.s. GRPO on math

a) GR-REINFORCE



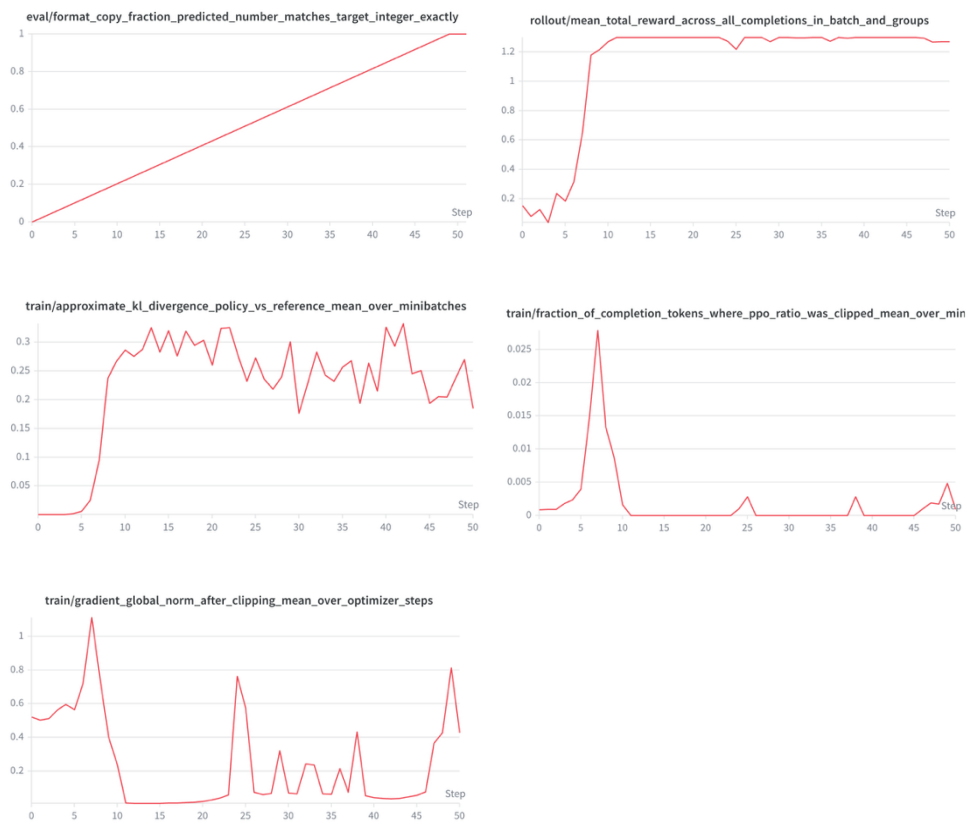
b) GRPO



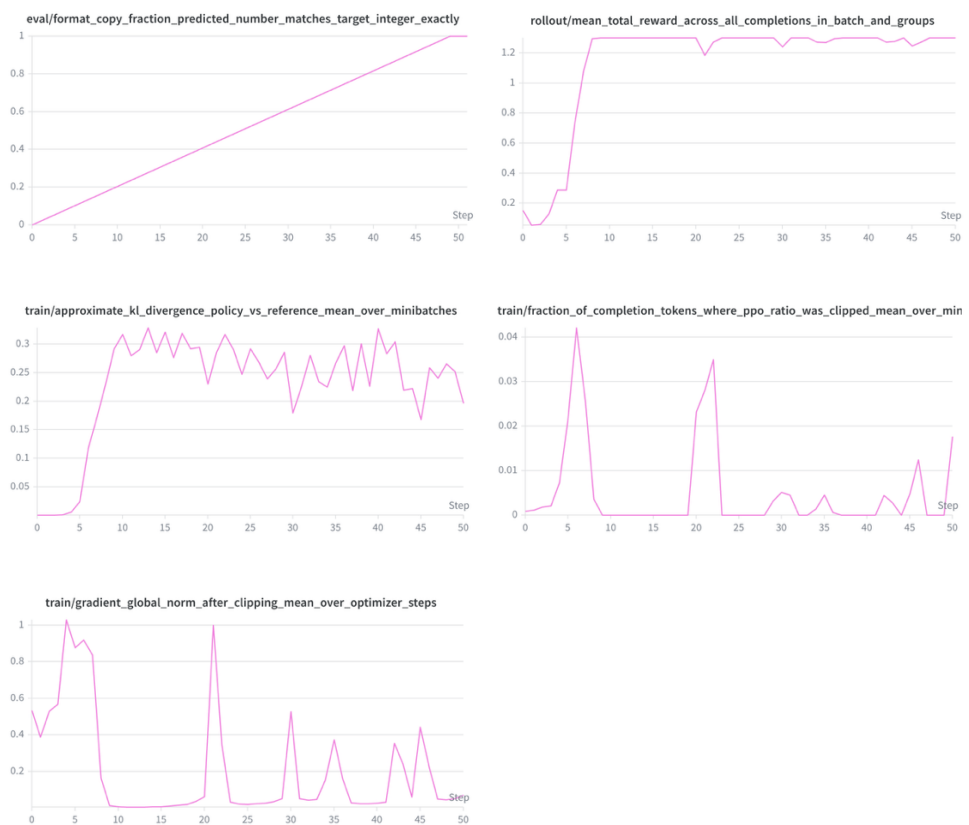
- c) Both methods improved over the baseline on math_hard, but GRPO learned faster and reached a higher final evaluation score. Over the first 200 iterations, GR-REINFORCE increased only modestly from the baseline, while GRPO improved more clearly by the same point. This comparison is meaningful because the provided commands keep most training settings the same, so the main algorithmic difference is that GRPO reuses each rollout batch for two PPO epochs, whereas GR-REINFORCE is strictly single-pass and on-policy. The WandB curves therefore suggest that GRPO is more sample-efficient on this task.

4. GRPO ablations on format copy

a) Results from original GRPO:



b) More PPO epochs (ppo_epochs = 3)



Increasing ppo_epochs from 2 to 3 did not meaningfully improve final performance on format_copy, as both settings quickly reached near-maximal rollout reward and achieved an eval exact match of 1.0. However, the ppo_epochs=3 run showed a somewhat higher clipped-token fraction and more volatile gradient norms, suggesting that the policy was updated more aggressively and moved farther from the behavior policy during rollout reuse. The approximate KL stayed in a similar range, so training remained stable overall. This suggests that the extra PPO pass increased optimization intensity, but on this easy task the default setting was already sufficient, so the added reuse did not translate into a clear performance gain.

c) Small KL coefficient (kl_coef = 0.01)



Reducing kl_coef from 0.05 to 0.01 did not dramatically increase the observed KL or clipped-token fraction, and the training and evaluation reward stays approximately the same, which suggests that on this easy task the optimization was already largely controlled by PPO clipping and the task's rapid saturation, rather than by the KL penalty alone.

d) Large KL coefficient (kl_coef = 0.40)



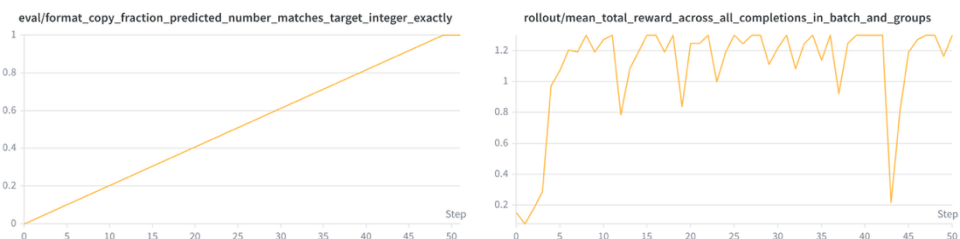
Increasing `kl_coef` from 0.05 to 0.40 made training more conservative overall. Compared with the default run, the larger-KL run had a lower clipped-token fraction and more conservative KL divergence especially early in training, which is consistent with a stronger reference-policy constraint. At the same time, the rollout reward was somewhat lower and noisier, even though the run still achieved an eval exact match of 1.0 in the end. This suggests that on `format_copy`, a much larger KL penalty did not improve final performance and instead mainly restricted optimization without providing a clear benefit.

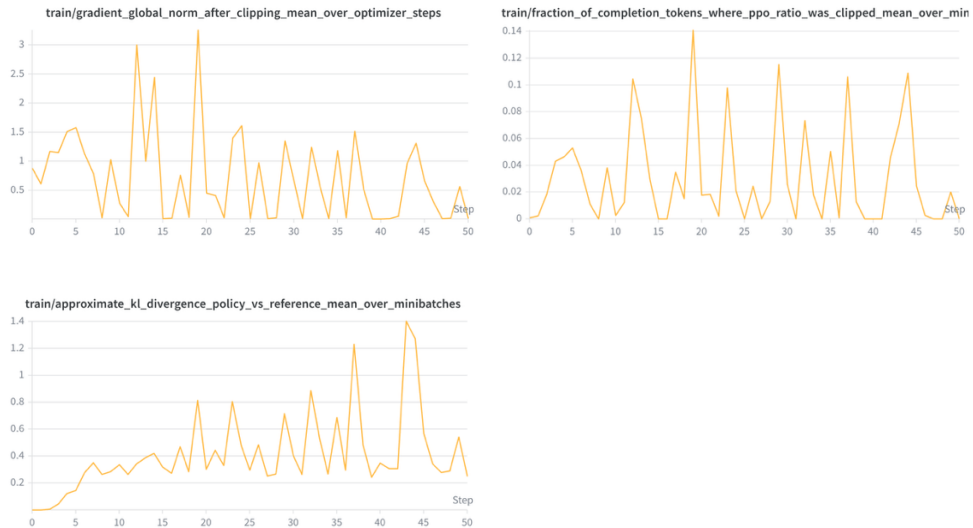
e) Stricter clip (`clip_eps = 0.01`)



Reducing clip_eps from 0.2 to 0.01 made clipping more aggressive, as shown by the substantially larger clipped-token fraction and smaller gradient norm, especially early in training. This is consistent with a much narrower PPO clipping range, where even small deviations in the policy ratio are clipped. The run still reached an eval exact match of 1.0, since format_copy is an easy task, but reward improved more conservatively and the optimization was more restricted overall. The approximate KL is also restricted by stricter clipping, as it appears smaller especially late in training

f) Step every minibatch (grad_accum_steps = 1)





Setting `grad_accum_steps` to 1 made optimization much more aggressive and less stable. Compared with the default setting, this run showed much larger spikes in approximate KL, clipped-token fraction, and gradient norm, and the rollout reward fluctuated far more during training. This is consistent with the fact that stepping after every minibatch greatly increases the number of optimizer updates per rollout and makes the updates noisier, as the updated policy drifts much further away from the rollout policy. Even so, `format_copy` is easy enough that the run still reached an eval exact match of 1.0 in the end. So on this task, stepping every minibatch did not improve final performance, but it clearly made training less stable.

- g) *Conclusion:* In my GRPO ablations on `format_copy`, all settings eventually reached an eval exact match of 1.0, so the main differences were in training stability rather than final accuracy. Increasing `ppo_epochs` made updates more aggressive but did not improve final performance. Lowering `kl_coef` had only a small effect, likely because PPO clipping and the simplicity of the task already limited policy drift. Larger KL coefficients made training more conservative without helping performance. The most important settings were `grad_accum_steps` and `clip_eps`: setting `grad_accum_steps=1` made training much noisier, while smaller `clip_eps` increased the clipped-token fraction and over-constrained learning. Overall, the default GRPO setting gave the best balance between stability and performance.

5. One informative example came from the GR-REINFORCE run (the 4th example shown below): for the same geometry prompt, most sampled completions produced the correct boxed answer `\boxed{2}`, but one completion produced the wrong answer `\boxed{18/7}`. That sample still received the small formatting reward because it used the boxed format, but its total reward was much lower, and its advantage was strongly negative. This shows that the model had learned the

desired output format, while reward and group-relative advantage still clearly separated mathematically correct and incorrect completions.

Another interesting example came from the GRPO run near the end of training: all 8 sampled completions for the logged prompt produced the same correct boxed answer `\boxed{13}` and received identical rewards. As a result, their group-normalized advantages were all zero. I found this informative because it shows that once GRPO becomes very consistent on an easy prompt, the group can lose its relative learning signal even though the outputs are already correct.

[illegible]